

DEMO GUIDE

BIG-IP NEXT FOR KUBERNETES 2.4 | AI INFERENCE ROUTING

Same GPUs, smarter routing

BNK 2.4 with F5 Endpoint Picker vs NVIDIA Dynamo EPP

How to run the UDF value demo in about 25 minutes:
the lab, the live run, the results, and what is under the hood.



Introduction

This guide shows how to run the **BNK 2.4 + Dynamo 1.5 Integration – Value Demo** in UDF. It is written for SEs who know BIG-IP and networking well. You do not need Kubernetes, NVIDIA Dynamo or GPU knowledge.

After this guide you should be able to:

1. Explain the lab in about two minutes.
2. Run the demo from one Grafana dashboard.
3. Explain where BNK helps and why.
4. Answer the obvious questions; Module 4 has an FAQ.

Demo length: about 25 minutes, including one live run.

The story in one sentence: on the same GPUs with the same traffic, BNK 2.4 with the F5 Endpoint Picker (F5 EPP) delivers the most value of the three routers, because it sends each request to a GPU worker that already has its prompt cached and is not overloaded.

Set expectations correctly: against a classic load balancer the gain is large. Against NVIDIA's own KV-aware routing (Istio + NVIDIA Dynamo EPP) the gain is real but smaller, mostly for the slowest users. Module 4 explains this.

Modules

#	Content	Time
0	Before the demo: start, readiness, pre-run	about 20 min from a cold start, before the customer joins
1	The lab in two minutes	5 min
2	Results: mixed GPU fleet (pre-run, preset A)	7 min
3	Live run: identical GPUs at peak hour (preset D)	8 min
4	What it means, and FAQ	5 min
5	Under the hood: demo tooling, BNK provisioning, F5 EPP configuration, KV events	reference
A	Appendix: presets, custom runs, troubleshooting, lab facts	reference

Conventions

- **Click** means a button on the Grafana dashboard.
- Screenshots use Grafana's light theme for printing; the live dashboard is dark by default. Layout and numbers are the same.
- **What you should see** boxes give the expected result. Numbers are from lab runs on 1 October 2026; your numbers will differ by roughly 10–20 % from run to run.
- Terms used in this guide:
- **TTFT** = time to first token: how long a user waits before the answer starts to appear.
- **p50 / p95 / p99**: the typical user / the slowest 5 % / the slowest 1 %.
- **KV cache**: the GPU memory where a worker keeps prompts it has already read.
- **EPP**: endpoint picker, the component that picks a worker for each request.

Module 0 — Before the demo

Do this module **before the customer joins**. From a cold start it takes about 20 minutes: about 10 minutes for the lab to get ready, a minute for the health check, then about 7–8 minutes for the pre-run.

Lab 0.1 — Deploy or start the lab

1. Log in to **UDF**.
2. Deploy the Blueprint **BNK 2.4 + Dynamo 1.5 Integration – Value Demo**, or start your existing deployment of it.

The lab prepares itself after every start; there are no manual steps. From a cold start this takes **about 10 minutes**.

Important: try to avoid **Force Stop** on this deployment. If nodes are still shutting down, a forced stop may lose the node disks, and the lab would then need a full rebuild. Use a normal stop.

Lab 0.2 — Open the dashboard and wait for "Lab ready"

1. In UDF, open the deployment's **Grafana** access method and log in with the Grafana admin account (ask the lab owner for the password if it is not in the blueprint description).
2. The home dashboard is **AI Inference Routing — BNK 2.4 demo**.

What you should see: the *Lab* tile at the top right shows **Lab ready – GPUs idling** when the lab has finished starting.

Lab 0.3 — Run the health check

Even when the dashboard shows *Lab ready*, run the health check once before a customer demo to be sure.

1. In UDF, open **Jumphost → Access → Web Shell**.
2. Run:

```
bash ~/lab/path-check.sh
```

What you should see: every line green, and the last line **PATH CHECK: OK**. The check confirms that every router in the lab answers, that each has two network paths, and that no AI traffic uses the management network.

```

ubuntu@jumpphost:~$ bash ~/lab/path-check.sh
1. client route per router (must go via router1 10.1.30.5, not mgmt ens5)
  ● haproxy 172.16.210.80: 172.16.210.80 via 10.1.30.5 dev ens6 src 10.1.30.101 uid 1000
  ● istio-epp 172.16.210.81: 172.16.210.81 via 10.1.30.5 dev ens6 src 10.1.30.101 uid 1000
  ● bnk-f5-epp 172.16.201.110: 172.16.201.110 via 10.1.30.5 dev ens6 src 10.1.30.101 uid 1000
2. router1 ECMP paths
  ● haproxy: 2 paths
  ● istio-epp: 2 paths
  ● bnk-f5-epp: 2 paths
3. HTTP /v1/models from client
  ● haproxy: 200
  ● istio-epp: 200
  ● bnk-f5-epp: 200
3a. one chat completion per router (real inference through the router)
  ● haproxy: chat 200
  ● istio-epp: chat 200
  ● bnk-f5-epp: chat 200
3b. lab-health guards (GR0, Istio DestinationRule, TMM, kv-bridge, licence, demo-api)
  ● node4 GR0 off on DPU ports
  ● node5 GR0 off on DPU ports
  ● Istio DestinationRule for the Dynamo EPP present
  ● BNK TMM pods: 2/2 running (one per DPU)
  ● kv-bridge running (Dynamo 1.5 KV events -> F5 EPP)
  ● BNK licence: Active
  ● demo-api answers (Grafana buttons)
4. no request traffic on management NICs of node2/node3 (tcp 8000/80, 20 requests per router)
  ● node2 ens5: 0 request/VXLAN packets
  ● node3 ens5: 0 request/VXLAN packets
PATH CHECK: OK

```

Figure 1. Health check output

If the last line is not **PATH CHECK: OK**, do not start the demo. See Appendix A.3.

Lab 0.4 — Pre-run preset A

Preset A takes about 7–8 minutes, which is too long to watch live. Run it now, so the results are on screen when the customer joins.

The **Routers** selector at the top of the dashboard (in the variables bar) decides which routers take part in a run:

- **Standard - HAProxy + Istio + BNK**: the default; use it for customer demos.
- **Istio vs BNK - fastest**: only the two KV-aware routers; the run is shorter.

Other options in the selector are lab extras; you do not need them for the demo. Routers that are not in the run show no bars in the result panels and no progress gauge.

1. Check that **Routers** is set to **Standard - HAProxy + Istio + BNK**.
2. **Click A. Mixed GPU fleet.**
3. A small confirmation tab opens and closes. The *Lab* tile changes to **Cooking tokens....**
4. Wait until the progress gauges of all routers in the run show **100 %** and the *Lab* tile shows **Served hot.**

What you should see during the run: the *Now* tile shows which router is being tested and how many requests are in flight. The routers run one after another: HAProxy, Istio + Dynamo EPP, BNK 2.4.

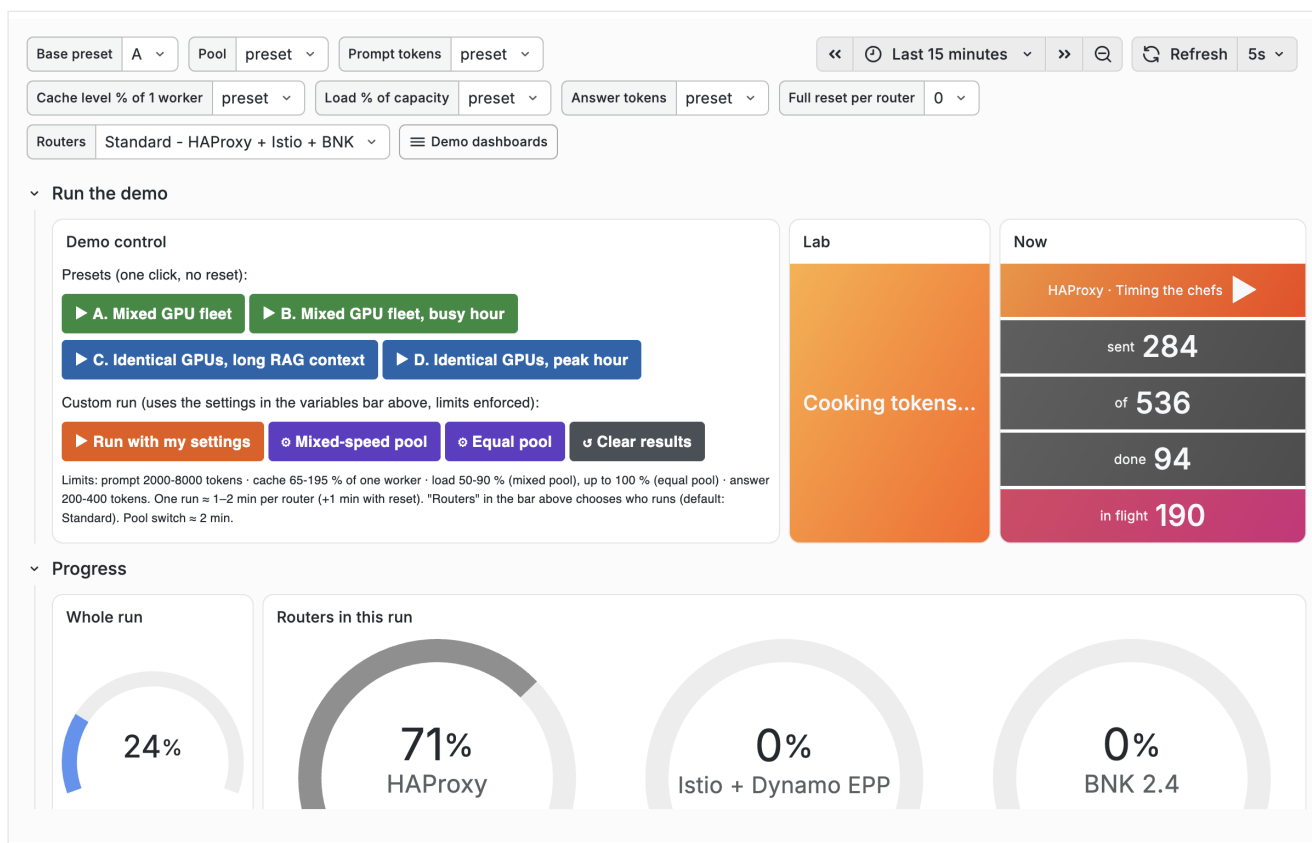


Figure 2. Preset A running — HAProxy phase

Tip: keep this browser tab open. The dashboard shows the **latest run per router**, so the preset A results stay visible until you start the next run.

Module 1 — The lab in two minutes

This module is talk only. Use the two diagrams.

Lab 1.1 — Why AI routing is different

Say, in this order:

1. An LLM request has two phases. First the GPU reads the whole prompt (**prefill**). Then it writes the answer token by token.
2. Prefill is expensive for long prompts. Typical enterprise apps send a long shared prompt with every request: a system prompt, RAG documents, tool definitions, chat history.
3. If the GPU already holds that prompt in its **KV cache**, most of the prefill is skipped and the first token comes back much sooner.
4. So **which GPU gets the request** decides how long the user waits. A GPU with the prompt cached and a short queue answers fast. A GPU without the cache, or with a long queue, answers slowly.
5. A classic load balancer (round robin, least connections) sees neither the cache nor the queue inside the GPU worker.

Lab 1.2 — The three routers

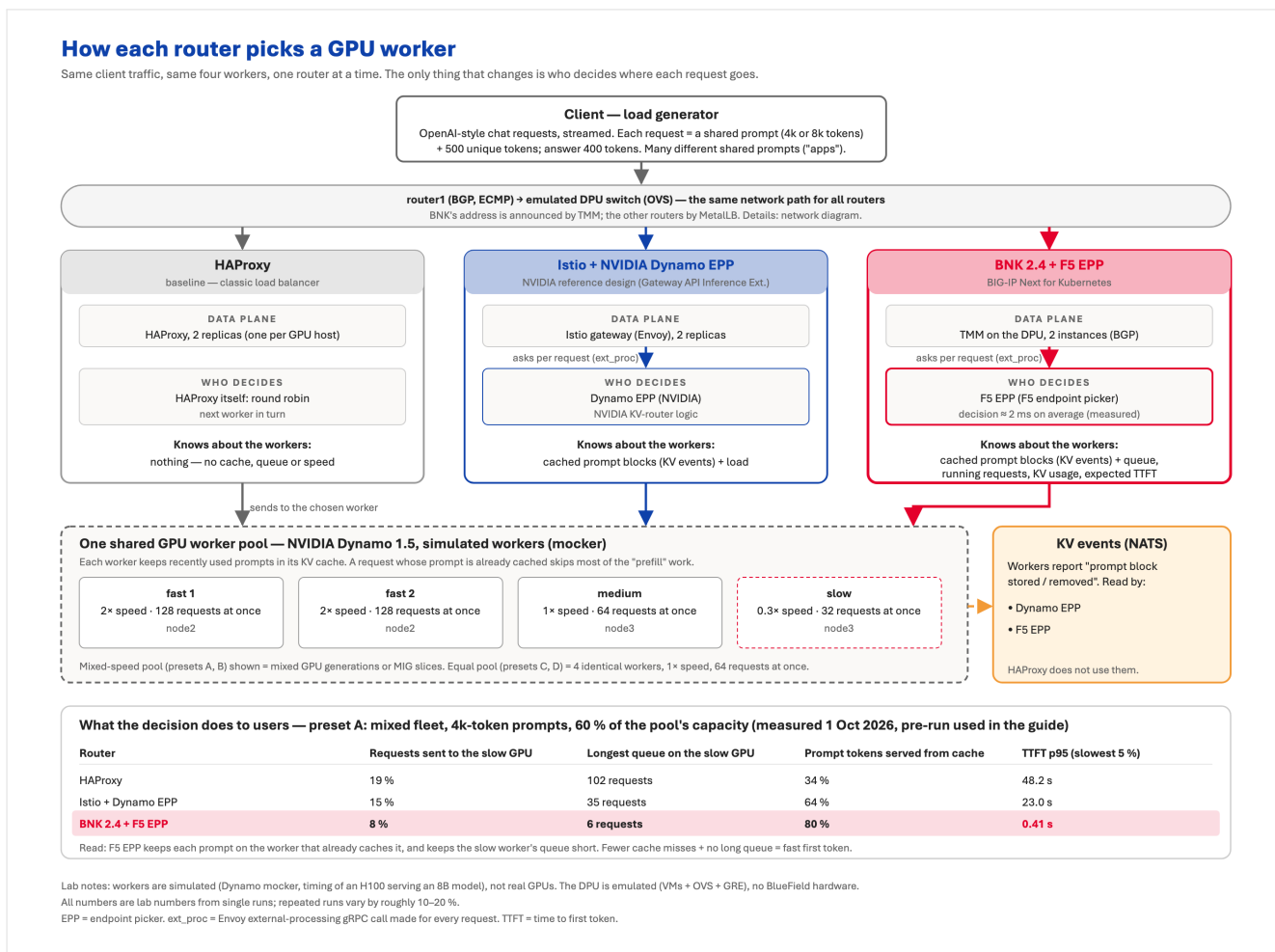


Figure 3. How each router picks a GPU worker

All three routers get the same traffic and use the same four GPU workers, one router at a time. The only difference is **who decides where each request goes**:

Router	Who decides	What it knows about the workers
HAProxy	HAProxy, round robin	Nothing: no cache, no queue, no speed

Router	Who decides	What it knows about the workers
Istio + NVIDIA Dynamo EPP	NVIDIA's endpoint picker, called by the Istio gateway for every request	Cached prompt blocks + load
BNK 2.4 + F5 EPP	F5 EPP, called by TMM for every request	Cached prompt blocks + queue, running requests, KV usage, predicted TTFT

The workers report which prompt blocks they store or drop (**KV events**). Dynamo EPP and F5 EPP both read these events; HAProxy does not.

Key point for the customer: Istio + Dynamo EPP is NVIDIA's own reference design for Kubernetes. We compare BNK against NVIDIA's best, not only against a classic load balancer.

Note: the lab also contains NVIDIA's Dynamo router (built into the Dynamo frontend). It is not part of the standard demo; if needed, add it to a run with the **Routers** selector at the top of the dashboard.

Lab 1.3 — Where BNK sits in the network

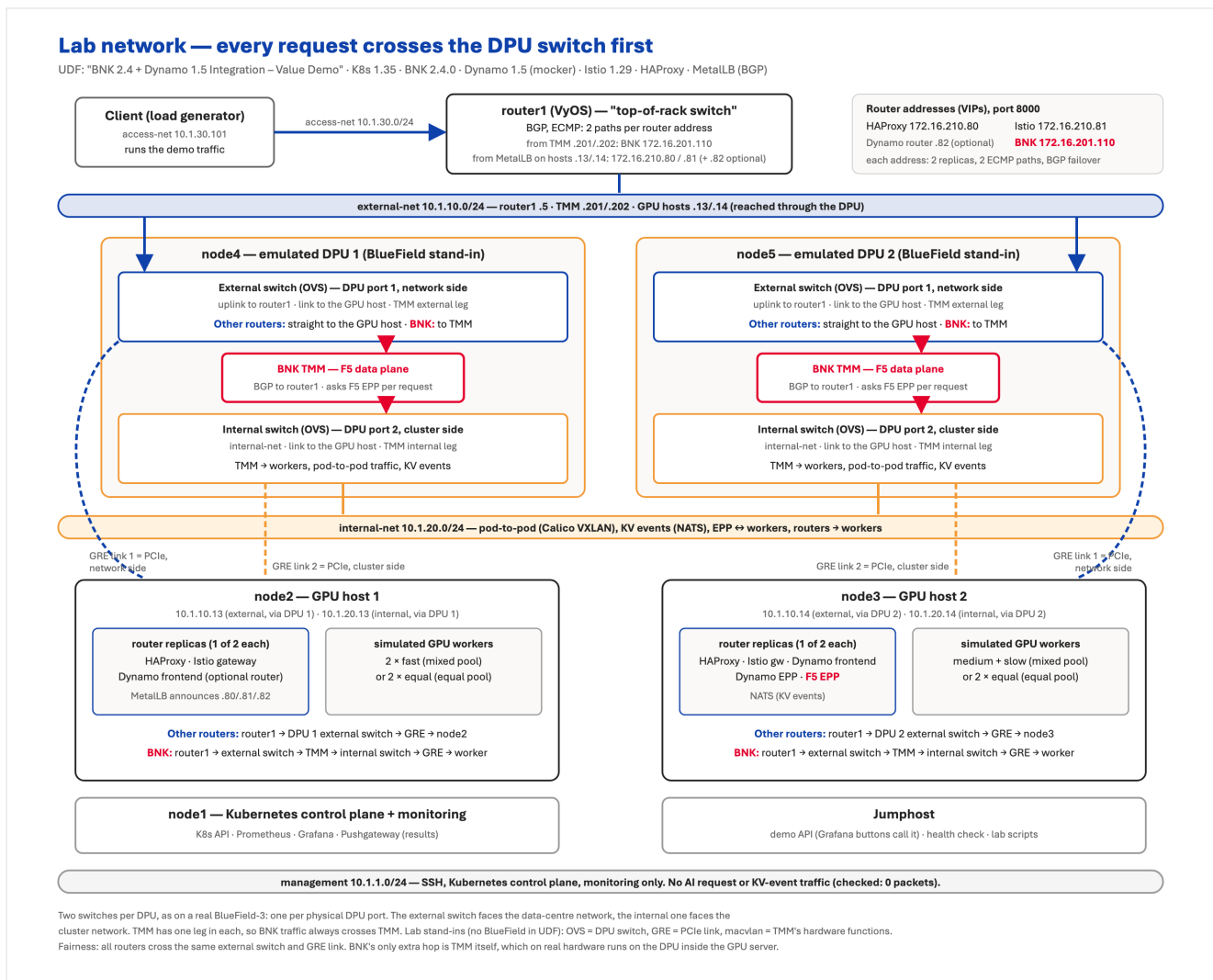


Figure 4. Lab network

Say:

1. The client reaches the GPU cluster through a top-of-rack router (router1), with BGP and two equal-cost paths per router address.
2. Every request first enters a **DPU** in front of the GPU server. In this lab the DPU is emulated: a VM with two software switches (OVS), wired like F5's BlueField-3 design. One switch faces the data-centre network, the other faces the cluster; TMM sits between them.

3. **BNK's TMM runs on the DPU.** For every request, TMM asks F5 EPP which worker should get it. The decision takes about 2 ms on average.
4. The other routers enter through the same router and the same DPU switch. The network path is the same for every router, so the comparison is fair.

Module 2 — Results: mixed GPU fleet (preset A)

Scenario: a pool of **mixed GPUs**, for example different GPU generations or MIG slices. Two fast workers (2× speed), one medium (1×), one slow (0.3×). Many apps, each with a 4,000-token shared prompt. The pool runs at 60 % of its capacity.

Lab 2.1 — Time to first token

1. Scroll to **TTFT — time to first token**.

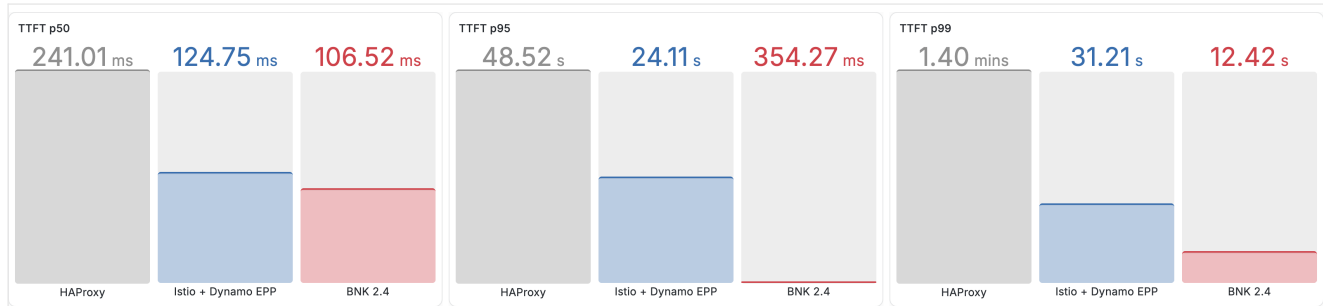


Figure 5. TTFT after preset A

What you should see (3 Oct run, approximately):

Router	TTFT p50 (typical user)	TTFT p95 (slowest 5 %)
HAProxy	241 ms	48.5 s
Istio + Dynamo EPP	125 ms	24.1 s
BNK 2.4	107 ms	350 ms

Say: "The typical user (p50) sees a similar wait with both KV-aware routers. The difference is in the slowest 5 % of users: 24 s with Istio + Dynamo EPP, 0.35 s with BNK."

Lab 2.2 — Why: cache hits and the slow GPU

1. Scroll to **Cache, ITL, errors and worker distribution**.

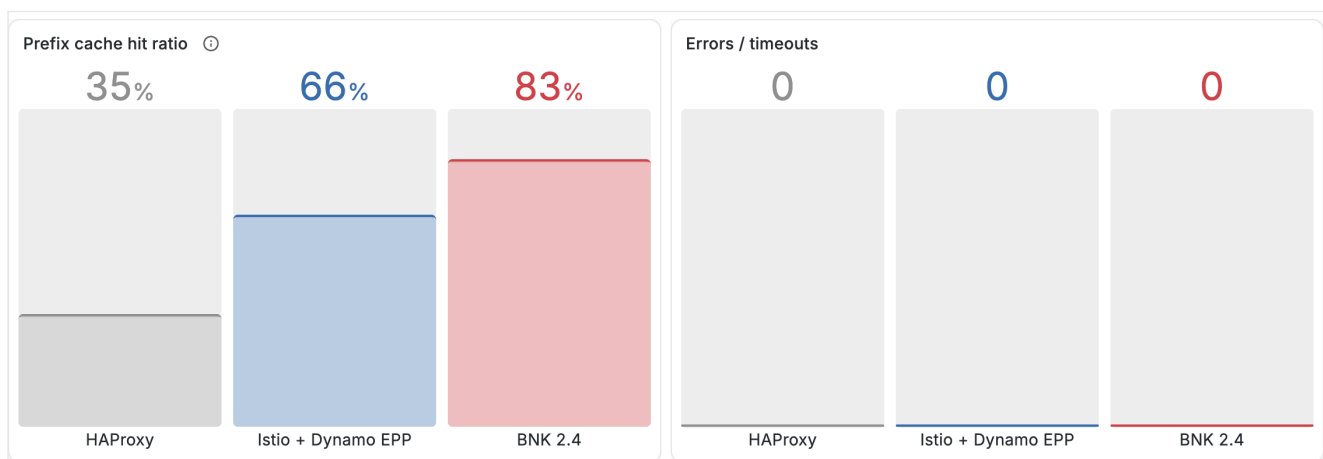


Figure 6. Prefix cache hit ratio and errors

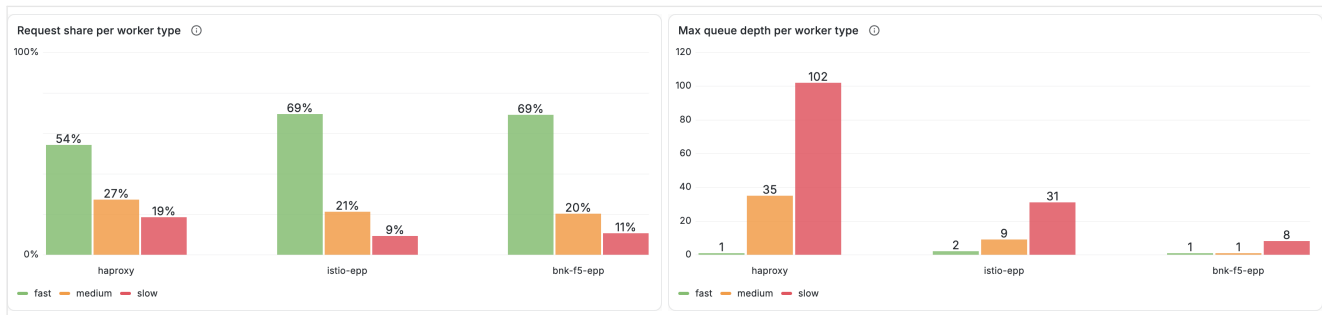


Figure 7. Request share and longest queue per worker type

To show the same effect over time, scroll to **Live worker metrics** → **Queue depth per worker** and set the time range (top right) to cover the pre-run, for example *Last 30 minutes*. The coloured bands below were added for this guide; the dashboard does not show them.

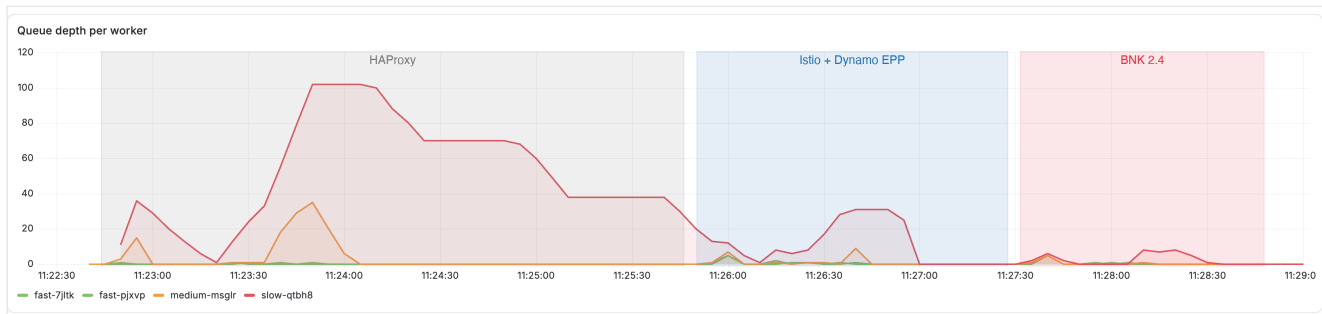


Figure 8. Queue depth per worker during preset A — the slow GPU (red line) queues up to about 100 requests with HAProxy, about 30 with Istio + Dynamo EPP, under 10 with BNK

What you should see:

Router	Prompt tokens served from cache	Requests sent to the slow GPU	Longest queue on the slow GPU
HAProxy	35 %	19 %	102
Istio + Dynamo EPP	66 %	9 %	31
BNK 2.4	83 %	11 %	8

Explain the chain of cause and effect:

1. F5 EPP keeps each shared prompt on a worker that already caches it, so fewer prompts are read again from scratch (higher cache hit).
2. F5 EPP also sees that the slow worker is slow and busy, so it sends that worker fewer requests.
3. The slow worker's queue stays short, so no user waits behind a long queue.
4. Together this removes the long tail: the slowest 5 % of users get their first token in well under a second.

Errors and timeouts should be **0** for every router.

Lab 2.3 — Throughput

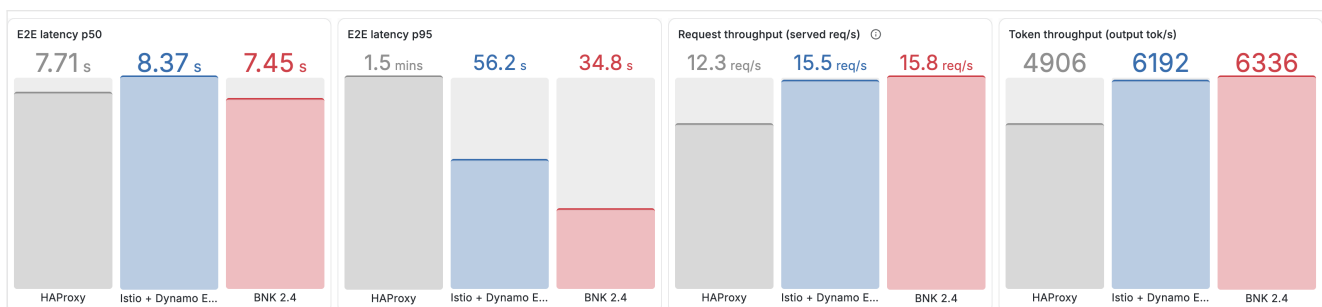


Figure 9. E2E latency and throughput

What you should see: served requests per second are close for the two KV-aware routers (Istio + Dynamo EPP 15.5, **BNK 15.8** req/s), and lower for HAProxy (12.3).

Say honestly: "At this load, the gain is mainly **latency for the slowest users**, not much more throughput. The same GPUs serve about the same number of requests; BNK serves them with far fewer long waits."

Module 3 — Live run: identical GPUs at peak hour (preset D)

Scenario: four **identical** GPUs, 4,000-token shared prompts, the pool at **100 %** of its measured capacity. There is no slow GPU to avoid, so the only difference between the routers is where they place the prompts.

Lab 3.1 — Start preset D

1. Check that **Routers** is set to **Standard - HAProxy + Istio + BNK**. If time is short, **Istio vs BNK - fastest** skips HAProxy and shortens the run.
2. **Click D. Identical GPUs, peak hour.**
3. The lab first switches the worker pool from mixed to identical GPUs. The *Lab* tile shows **Replacing GPUs...** for about 2 minutes.

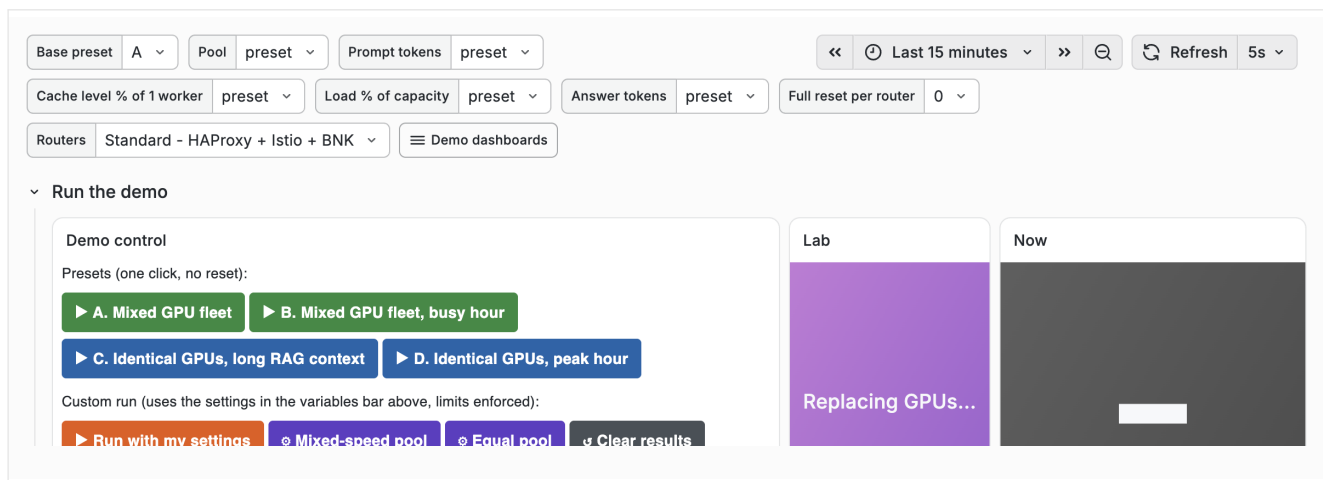


Figure 10. Lab switching the worker pool

4. Then the three routers run one after another, under 1 minute each: about 3 minutes in total (measured), plus the 2 minutes of GPU switching when the pool has to change.

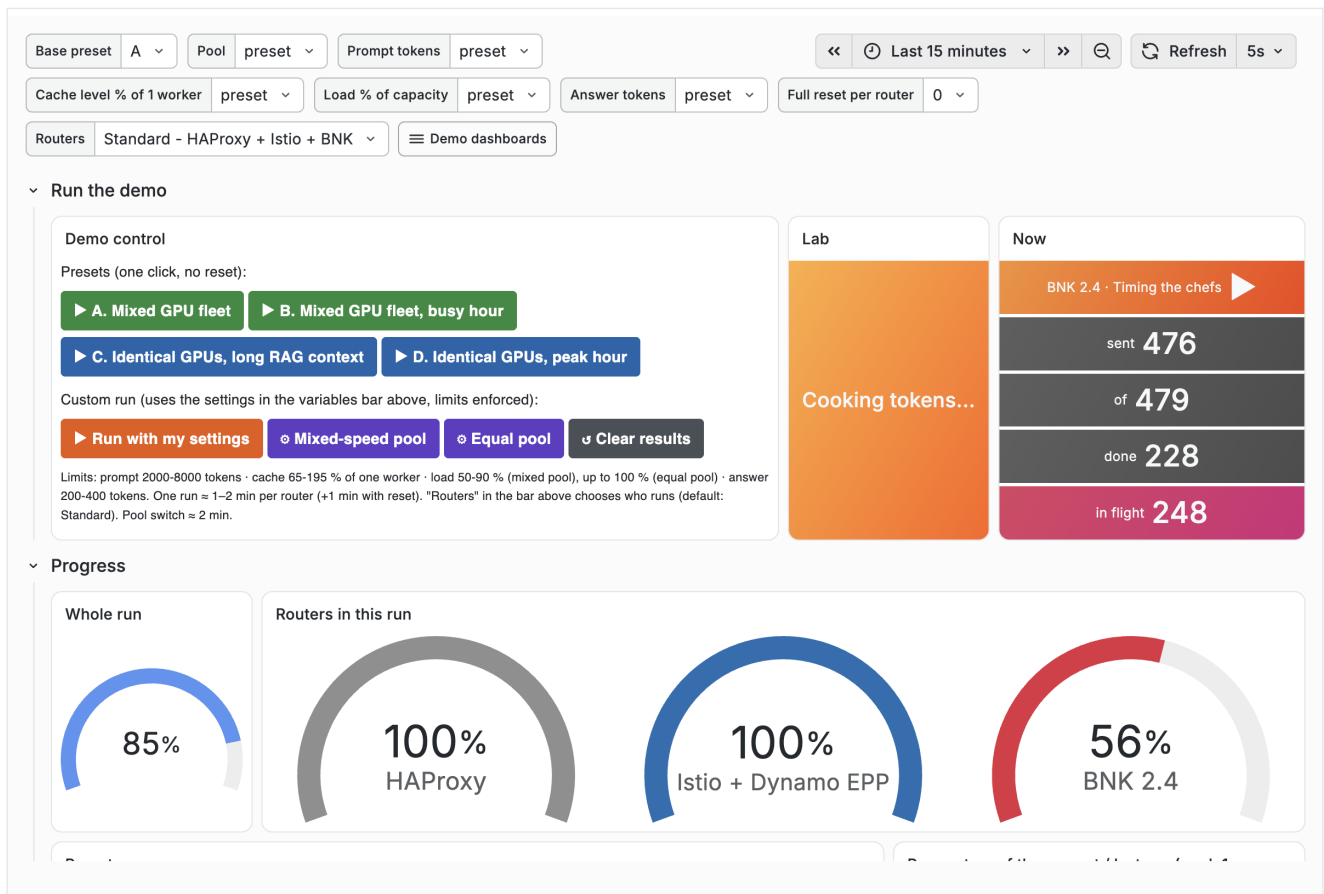


Figure 11. Preset D running — BNK phase, the earlier routers done

Use the waiting time to explain Module 1 again, or to take questions.

Lab 3.2 — Watch the queues live

1. Scroll to **Live worker metrics** → **Queue depth per worker**.

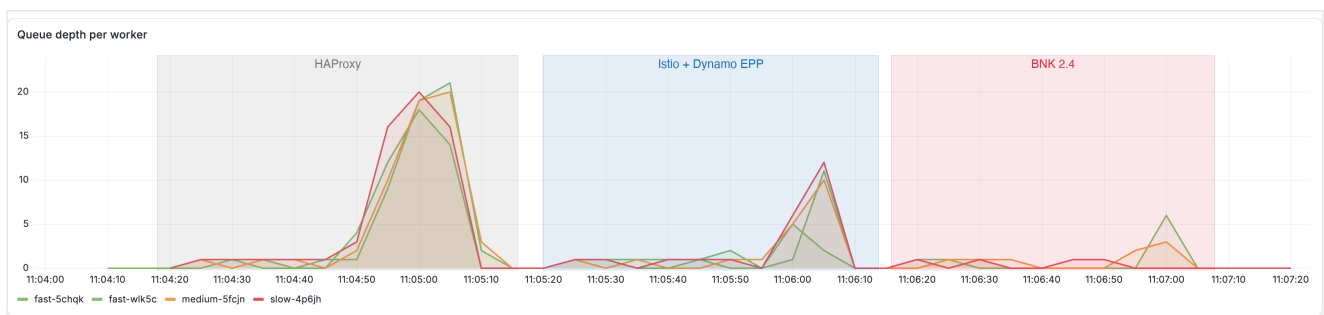


Figure 12. Queue depth per worker during preset D (router bands added for this guide)

What you should see: during the HAProxy phase, all four workers build queues of about 20 requests. With Istio + Dynamo EPP the longest queue peaks at roughly 10–12. During the BNK phase the longest queue stays at a few requests.

Lab 3.3 — Read the result

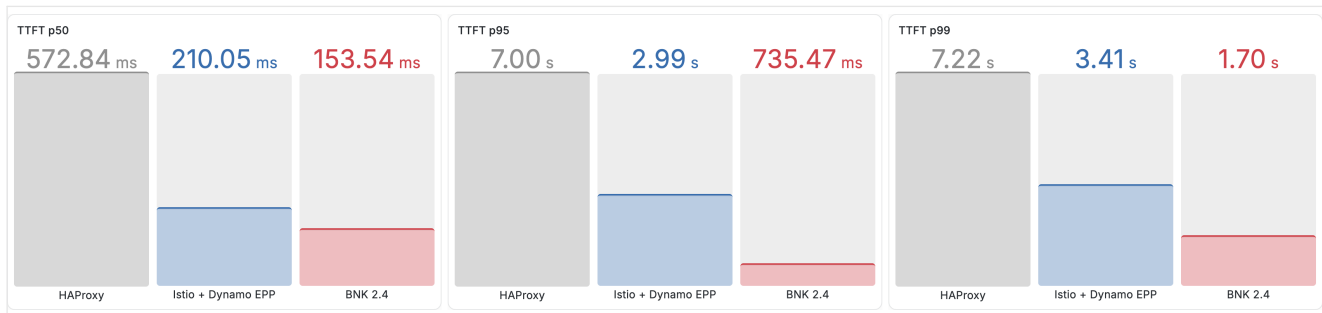


Figure 13. TTFT after preset D

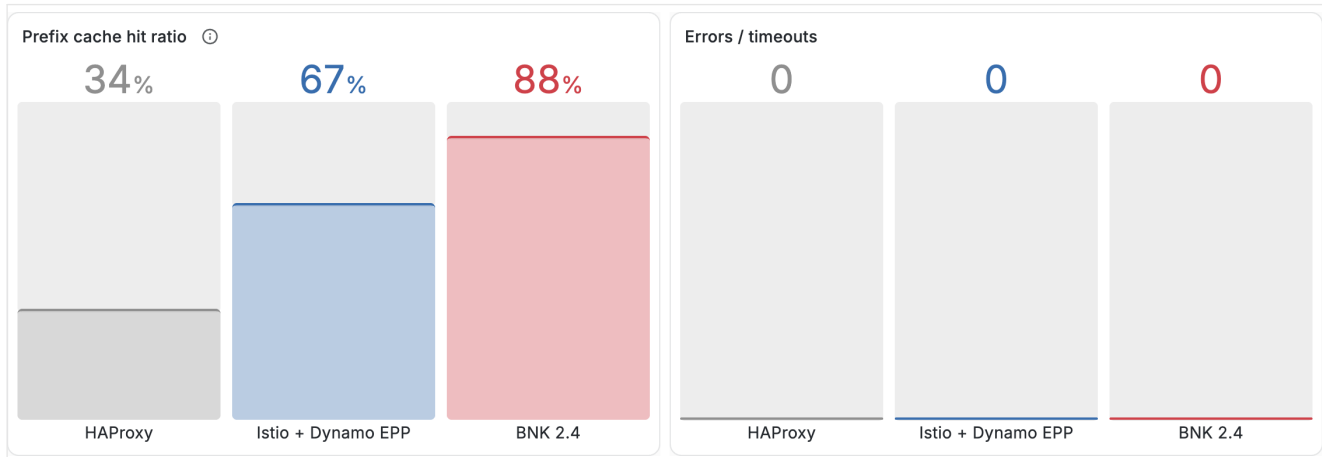


Figure 14. Cache hit and errors after preset D

What you should see (3 Oct run, approximately):

Router	TTFT p50	TTFT p95	Cache hit	Served req/s
HAProxy	573 ms	7.00 s	34 %	11.4
Istio + Dynamo EPP	210 ms	2.99 s	67 %	13.3
BNK 2.4	154 ms	0.74 s	88 %	13.9

Say: "Even with identical GPUs, at full load, placing prompts where they are cached matters. BNK serves more of the prompt from cache, so the slowest users wait less." Then add honestly: "Throughput is almost the same (13.9 vs 13.3 req/s). The gain is in waiting time, not in more GPU capacity."

Module 4 — What it means, and FAQ

Lab 4.1 — The F5 control point

Say:

1. Every request to the GPU cluster passes one point: the entry into the cluster. In a DPU-based design, BNK's TMM sits exactly there.
2. At that point BNK makes a per-request decision based on what the GPUs are doing (cache contents and load), not only on connections.
3. It is the same point where F5 already does what customers expect from an ADC: BGP and failover, TLS, security, multi-tenancy.
4. So the routing gain comes from a component the customer would place there anyway.

Lab 4.2 — How big is the gain? (say this before the customer asks)

Short answer: BNK gives the most value of the three routers, but it does not win every scenario, and against NVIDIA's own KV-aware routing (Istio + Dynamo EPP) the gain in the lab is moderate, not dramatic. Say this plainly; it builds trust, and the numbers support it.

Compared with	What the lab shows
Round robin (HAProxy)	Large, clear gain in every preset. Note: any KV-aware router gets most of this gain, including NVIDIA's.
Istio + Dynamo EPP — slowest users (TTFT p95/p99)	The main BNK gain. In our full test grid (94 comparable points, BNK vs Istio + Dynamo EPP), BNK had the better TTFT p95 on 81 points and a worse one on 12.
Istio + Dynamo EPP — cache hit	Usually higher with BNK (+10 to +17 percentage points in presets A–D).
Istio + Dynamo EPP — typical user (TTFT p50)	In the lab BNK was slightly slower on 70 of the 94 grid points, typically by about 15–35 ms. Most of this comes from the lab's emulated DPU: TMM runs on a separate VM behind a GRE tunnel. On a real BlueField DPU, TMM sits inside the GPU server, so in a real environment BNK's typical TTFT should probably be on par (expected, not measured). In presets A, B and D BNK's p50 is already the best.
Istio + Dynamo EPP — throughput	About the same in the lab (presets A–D: within about 7 %). The lab cannot show one real-world advantage: on a BlueField DPU, BNK's data plane runs on the DPU's own cores, so the GPU server's CPUs are not used for traffic handling. NVIDIA's path (Istio gateway + Dynamo EPP) runs on the GPU server's CPUs. In a real deployment this offload should give BNK an edge (expected, not measured in this lab).

Lab 4.3 — FAQ

Question	Answer
Are these real GPUs?	No. The workers are NVIDIA Dynamo's simulated workers ("mockers"), timed like an H100 serving an 8B model. The routing logic, the KV caches and the KV events are real Dynamo software. Treat the numbers as lab numbers, not a benchmark.
Is that a real BlueField DPU?	No. It is emulated with VMs, two software switches (OVS) per DPU and GRE tunnels, wired like F5's BlueField-3 reference design. All routers cross the same emulated DPU.
Why is BNK's typical latency sometimes a little higher?	In this lab TMM runs on a separate VM behind a GRE tunnel, which adds about 12–25 ms per request. F5 EPP's decision itself takes about 2 ms. On real hardware the DPU sits inside the GPU server, so we expect a smaller penalty there; this has not been measured.



Question	Answer
Is this F5 against NVIDIA?	No. BNK uses NVIDIA Dynamo workers and Dynamo's KV events. The comparison is about who makes the routing decision at the entry point to the GPU cluster.
Does it work with vLLM, SGLang or llm-d?	F5 EPP supports vLLM, SGLang and Dynamo engines (BNK 2.4 documentation). This lab tests Dynamo only.
Can the customer reproduce the numbers?	The method is: same workers, same traffic, one router at a time, full warm-up, 30-second measurement window. Results depend on prompt lengths, cache size and load; a customer test should use their own traffic shape.

Module 5 — Under the hood

This module is background for the SE. It explains how the demo is driven, what happens when the lab starts, how BNK 2.4 was provisioned, how F5 EPP is configured, and what a KV event looks like. It is not needed to run the demo, and it does not go into installation detail.

Lab 5.1 — How a demo run is started and measured

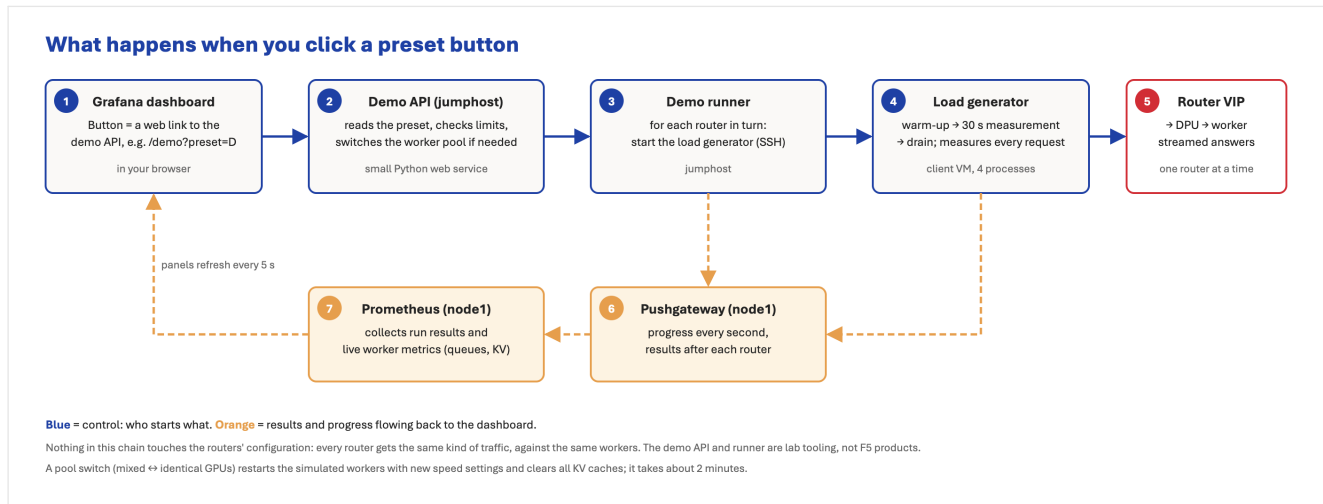


Figure 15. What happens when you click a preset button

Grafana buttons and the demo API. Each button on the dashboard is a plain web link to a small lab-only web service, the *demo API*, on the jumphost. For example, the preset D button opens:

```
https://<demo-api access URL>/demo?preset=D&reset=0
```

The demo API reads the preset (pool, prompt length, cache level, load), keeps the values inside fixed limits, and switches the worker pool first if the preset needs the other pool. It then starts the *demo runner*. The runner tests the routers chosen in the **Routers** selector (by default HAProxy, Istio + Dynamo EPP and BNK) one after another and pushes progress and results to Prometheus (through a Pushgateway); the dashboard reads them from there.

The load generator. For each router, the runner starts the load generator on the client VM. The generator:

1. Builds a fresh set of **shared prompts** ("apps"). Each request = one shared prompt (4,000 or 8,000 tokens, standing in for a system prompt or RAG context) + 500 unique tokens (the user's question). The answer is 400 tokens, streamed.
2. **Warm-up:** sends every shared prompt once, in the same order and at the same rate for every router, so each router places the prompts itself.
3. **Measurement, 30 seconds:** sends requests with random (Poisson) arrival times at a fixed rate, for example 15.8 requests/s in preset D (= 100 % of the pool's measured maximum request rate). It is an *open loop*: requests keep arriving even when the router is slow, as with real users.
4. **Drain:** stops sending and waits until every request has finished, so slow requests are counted too.

For every request, the generator records time to first token, time between tokens, total time, and how many prompt tokens the worker served from its cache (reported by the worker in the response). Four client processes share one pre-computed schedule, so every router gets exactly the same traffic shape.

How many shared prompts? The cache level sets it. For example, at cache level 195 % with 4,000-token prompts, the generator uses 181 shared prompts: together about $1.95 \times$ the KV cache of one worker. The prompts fit in the pool, but not in one worker, so the router's placement decides the cache hit rate.

Lab 5.2 — What happens when the lab starts

A script on the jumphost (`lab-boot.sh`, started automatically at every boot) makes the lab demo-ready:

1. Waits until Kubernetes, all 5 nodes, both BNK TMMs and the simulated GPU workers are up.
2. Rebuilds the HAProxy server list from the workers' current addresses (they change on every start).
3. Points the Grafana buttons at **this** deployment's demo API (each UDF deployment gets its own access URLs).
4. Starts the demo API.
5. Sends one test request through each router in the lab.

It reports its state to the dashboard's *Lab* tile: yellow while running, blue when ready, red if a step failed. The log is `~/lab-boot.log` on the jumphost. BNK runs on a 30-day evaluation licence; a separate start-up check renews it automatically when fewer than 5 days are left.

Lab 5.3 — How BNK 2.4 was provisioned (context only)

Step	What was done
Kubernetes	Upstream Kubernetes 1.35 on 5 UDF VMs (control plane, 2 GPU hosts, 2 emulated DPUs), installed with the F5 UDF lab installer.
BNK 2.4.0	Installed by the F5 Lifecycle Operator (FLO) with a CNEInstance resource (<code>manifestVersion: 2.4.0</code> , Gateway API enabled, BGP enabled). TMM runs as 2 instances on the 2 DPU nodes. Evaluation licence.
Enable F5 EPP	One extra line in the CNEInstance: <code>eppNamespaces: [dynamo-demo]</code> = the namespace where AI routing objects live.
Network	TMM peers with router1 over BGP and announces the BNK address 172.16.201.110.
Emulated DPU	Each DPU node has two OVS switches , as on a real BlueField-3: one per physical DPU port. The <i>external</i> switch joins the uplink to router1, the link to the GPU host's network side and TMM's external leg. The <i>internal</i> switch joins the cluster network, the link to the GPU host's cluster side and TMM's internal leg. Each switch reaches the GPU host over its own GRE tunnel, which stands in for the PCIe link.
AI routing objects	Four standard Kubernetes resources (below).

The AI routing setup, in the order the request uses it:

Resource	Standard or F5	What it says in this lab
Gateway <code>bnk-ai-gw</code>	Kubernetes Gateway API	class <code>f5-gateway-class</code> (= BNK), address 172.16.201.110, HTTP port 8000
HTTPRoute <code>qwen-bnk</code>	Kubernetes Gateway API	path <code>/v1</code> → InferencePool <code>qwen-f5</code>
InferencePool <code>qwen-f5</code>	Gateway API Inference Extension (same standard NVIDIA uses with Istio)	the Dynamo workers (selected by label), port 8000; endpoint picker = service <code>qwen-f5-epp</code> , port 9002; if the picker fails, requests are refused (<code>FailClose</code>)
F5EPP <code>qwen-f5-epp</code>	F5 resource (BNK 2.4)	<code>engine: dynamo, mode: aggregated, blockSize: 64</code> (must match the workers), tokenizer <code>Qwen/Qwen3-8B</code> , NATS address for KV events, worker metrics port 9090

The F5EPP resource makes BNK's operator deploy and configure the F5 EPP. TMM then calls F5 EPP for every request that matches the HTTPRoute.

Why two switches, not one: the two switches keep the data-centre network and the cluster network apart, so BNK traffic always passes through TMM between them. The other routers only use the external switch and go straight to the GPU host. One switch could do the same job only with VLANs or flow rules to keep the two networks separate; the lab follows F5's BlueField-3 reference design instead.

The key point for customers: **BNK uses the same Kubernetes standards as NVIDIA's reference path** (Gateway API + Inference Extension). Only the gateway class and the picker are F5.

Lab 5.4 — How F5 EPP chooses a worker

F5 EPP gives each worker a score from several signals, adds the scores using fixed weights, and a picker chooses the worker. These are the weights used in this lab (the default profile that BNK generated for the Dynamo engine):

Signal (scorer)	Weight	What it measures
Prefix cache	0.50	How much of this request's prompt the worker already holds in its KV cache (from KV events)
Token-weighted load	0.15	Work already queued on the worker, counted in tokens, not requests
Active KV blocks	0.10	KV blocks in use for running requests
Queue	0.10	Requests waiting on the worker
Running requests	0.10	Requests the worker is processing now
KV cache usage	0.03	How full the worker's KV cache is

Then the **picker** (F5's "HCL" picker) makes the final choice:

- If one worker clearly holds the prompt (**cache affinity**), it prefers that worker.
- If no worker holds it (a **cold** request), it picks the worker with the lowest **predicted time to first token**.
- If no worker is acceptable, it falls back to the least-bad choice.

Two protections exist but appear to be **off or set very loosely** in the generated defaults: an overload guard ("hot worker" exclusion) and a saturation limit. Their settings are not documented yet, so this is our reading of the configuration. The demo uses the defaults.

Why this matters for the result: half of the score is cache placement, and the other half is load in several forms. That is why BNK keeps prompts on the worker that caches them *and* sends less work to the slow GPU.

The weights are as generated. The lab changed only how F5 EPP connects to Dynamo 1.5: the worker metric names and the KV-event settings were aligned with Dynamo 1.5's format.

Lab 5.5 — What a KV event looks like

Every Dynamo worker publishes a short message on NATS (a lightweight message bus) whenever it stores or drops blocks of prompt in its KV cache. One block = 64 tokens. Routers that listen (for example F5 EPP and Dynamo EPP) keep a map of **which worker holds which prompt blocks**.

A real event captured in the lab, shortened:

```
subject: namespace.dynamo-demo-qwen-af53ff98.component.backend.endpoint.generate.kv-events
{
  "worker_id": 2263787179699545,
  "storage_tier": "device",          <- GPU memory
  "event": {
    "event_id": 7671,
    "data": {
      "stored": {                    <- or "removed": {"block_hashes": [...]}
        "blocks": [
          {"block_hash": 15386704986735564117, ...},
          {"block_hash": 3268014213677552799, ...},
          ... 7 more blocks
        ]
      }
    }
  }
}
```

Workers also send a small load message on the same bus:

```
subject: ...component.backend.endpoint.generate.kv_metrics
{ "worker_id": 2263787179699545, "kv_used_blocks": 10, "active_decode_blocks": null, ... }
```



How F5 EPP uses it: when a request arrives, F5 EPP tokenizes the prompt (about 2 ms), splits it into 64-token blocks, hashes them, and looks up which workers hold the longest matching run of blocks. That lookup feeds the 0.50 "prefix cache" score above.

Appendix

A.1 — All presets and custom runs

Preset	Pool	Prompt	Cache level	Load	One-line story
A	mixed-speed	4,000 tokens	195 %	60 %	Mixed GPU generations / MIG slices
B	mixed-speed	4,000 tokens	195 %	75 %	Same fleet, busy hour
C	equal	8,000 tokens	195 %	60 %	Identical GPUs, long RAG context
D	equal	4,000 tokens	195 %	100 %	Identical GPUs, peak hour

- **Cache level** = all distinct shared prompts together, as % of **one** worker's KV cache. 195 % means the prompts do not fit on one worker, so routing decides the cache hit rate.
- **Load** = % of the pool's measured maximum request rate.

Results from 1 October 2026 for presets B and C (single runs):

Preset	Router	TTFT p50	TTFT p95	Cache hit
B	Istio + Dynamo EPP	150 ms	33.7 s	67 %
B	BNK 2.4	133 ms	1.48 s	77 %
C	Istio + Dynamo EPP	146 ms	0.55 s	76 %
C	BNK 2.4	162 ms	0.21 s	93 %

In preset C, BNK's typical TTFT (p50) is 16 ms **slower** than Istio's, while its p95 and cache hit are better. Use C only if you are ready to explain the lab path delay.

Custom run: set *Pool*, *Prompt tokens*, *Cache level*, *Load* and *Answer tokens* in the variables bar at the top of the dashboard, then **click Run with my settings**. The limits are shown under the buttons and are enforced.

Which routers run: the **Routers** selector in the same variables bar applies to presets and custom runs alike:

Standard - HAProxy + Istio + BNK (default) or **Istio vs BNK - fastest**. Routers not in the run show no bars and no progress gauge.

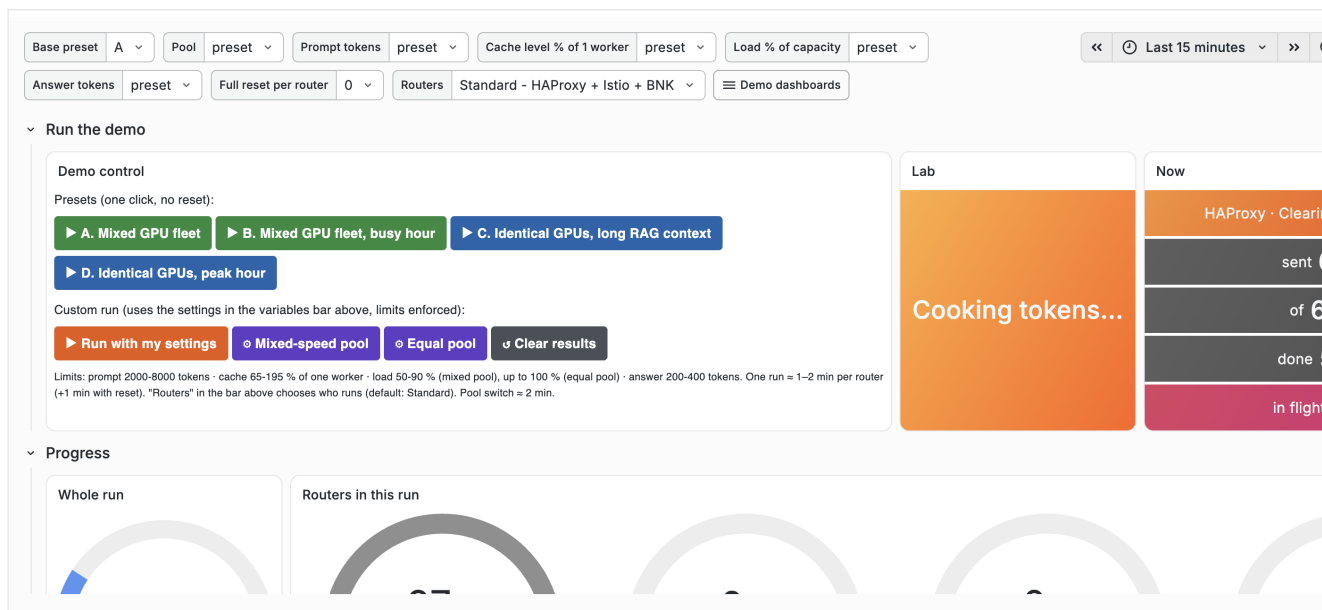


Figure 16. Variables bar and demo buttons

A.2 — Run a preset from the command line

On the jumphost web shell:

```
curl -s "localhost:5050/demo?preset=D&reset=0"
curl -s localhost:5050/status
```

`reset=1` restarts the routers and clears all KV caches before each router's run. This takes about 1 extra minute per router; it is not needed for a demo.

A.3 — If something looks wrong

Symptom	What to do
<i>Lab</i> tile stays yellow (Lab warming up) for more than 20 minutes	On the jumphost: <code>tail ~/lab-boot.log</code> shows the step it is waiting for.
<i>Lab</i> tile shows Burnt (red) right after start	Start-up found a problem: <code>tail ~/lab-boot.log</code> . Rerun it with <code>bash ~/lab/lab-boot.sh</code> (about 2 minutes).
<i>Lab</i> tile shows Burnt (red) after a run	On the jumphost: <code>curl -s localhost:5050/status</code> shows the error. Run <code>bash ~/lab/path-check.sh</code> .
Health check not OK	Do not demo. Restart the UDF deployment with a normal restart (try to avoid Force Stop), wait for <i>Lab ready</i> , and run the check again.
Panels empty	Wait for the run to finish; results appear per router as each phase ends. Check the time range (top right): <i>Last 15 minutes</i> .
Still broken	Contact the lab owner (see deployment description).

A.4 — Lab facts

Item	Value
Platform	UDF, 9 VMs: client, 2 routers (VyOS), jumphost, control plane, 2 GPU hosts, 2 emulated DPUs
Kubernetes	1.35
BNK	2.4.0 with F5 EPP (Dynamo engine); 30-day evaluation licence, renewed automatically at start
Workers	NVIDIA Dynamo 1.5 mocker; each about one H100 serving an 8B model, 371k-token KV cache
Mixed-speed pool	2 × fast (2× speed, 128 requests at once), 1 × medium (1×, 64), 1 × slow (0.3×, 32)
Equal pool	4 × 1× speed, 64 requests at once
Router addresses	HAProxy 172.16.210.80, Istio 172.16.210.81, BNK 172.16.201.110 (port 8000).
Traffic	Shared prompt + 500 unique tokens per request, 400-token answer, Poisson arrivals, 4 client processes
Routers in a run	Set by the Routers selector; default HAProxy, Istio + Dynamo EPP, BNK 2.4
Method per router	Warm-up (every shared prompt once) → 30 s measurement → drain. Same prompt set shape, new prompts per router.